Queues

Queues

A queue is a FIFO (**First-In, First-Out**) data structure in which the element that is inserted first is the first one to be taken out. The elements in a queue are added at one end called the REAR and removed from the other end called the FRONT.

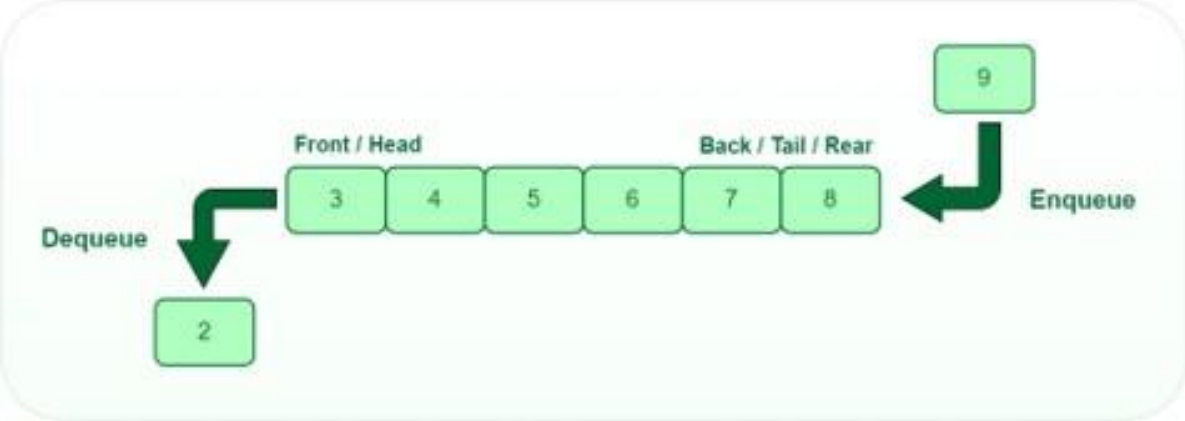
Example:

People standing outside the ticketing window of a cinema hall.
Luggage kept on conveyor belts
Cars lined at a toll bridge

Implemented: Using Arrays as well as Linked List

Types:

Linear Queue
Circular Queue
Double Ended Queue
Priority Queue



Operations:

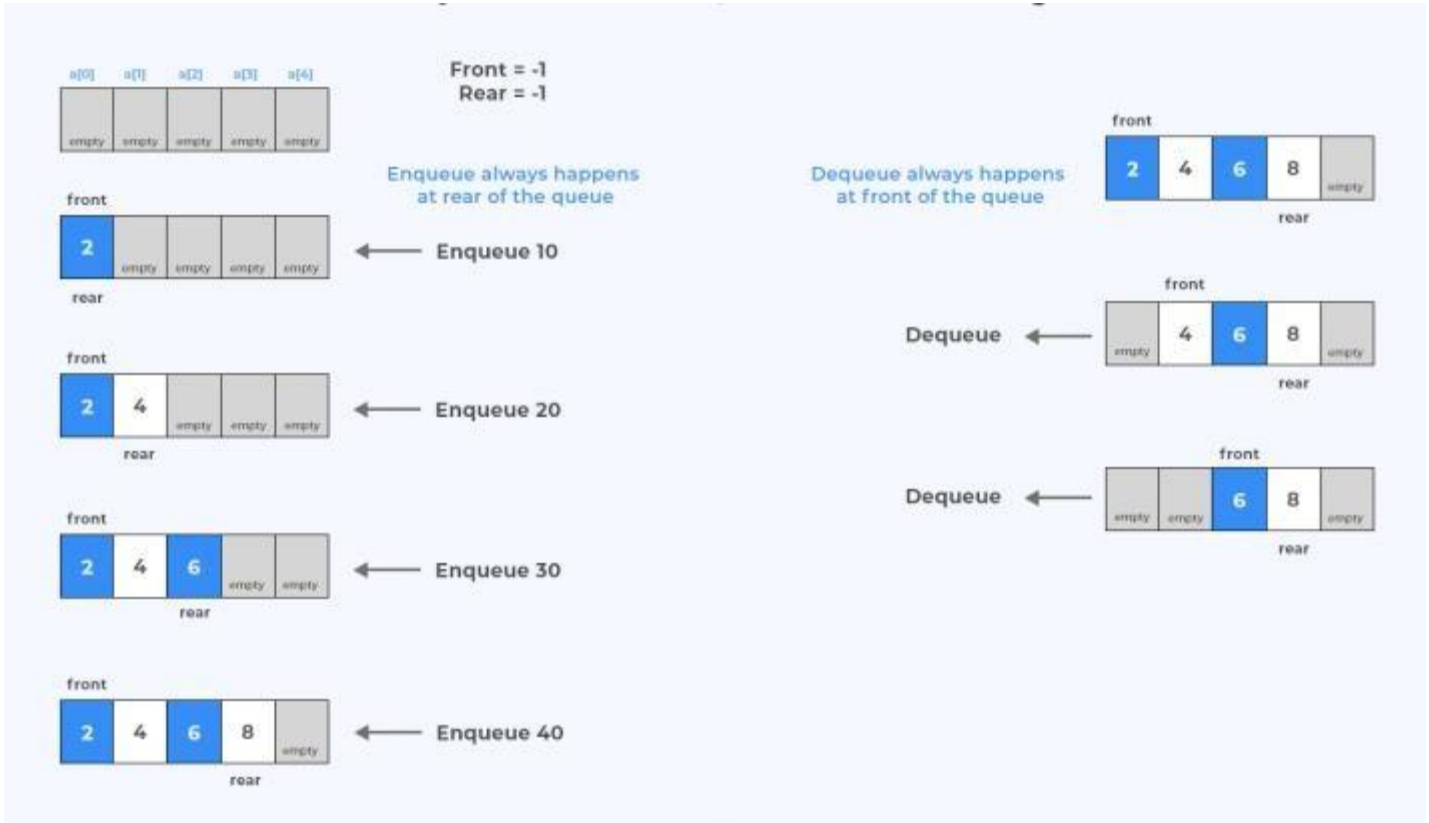
Insertion: Enqueue
Deletion Dequeue
Peek

Applications of Queue

- **Printers:** Queue data structure is used in printers to maintain the order of pages while printing.
- **Interrupt handling in computes:** The interrupts are operated in the same order as they arrive, i.e., interrupt which comes first, will be dealt with first.
- **Process scheduling in Operating systems:** Queues are used to implement round-robin scheduling algorithms in computer systems.
- **Switches and Routers:** Both switch and router interfaces maintain ingress (inbound) and egress (outbound) queues to store packets.
- **Customer service systems:** It develops call center phone systems using the concepts of queues.

VIPS
योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION
Vivekananda Institute of Professional Studies

Array Representation of Linear Queue



Queue Insertion

Implementation of Queue using array.

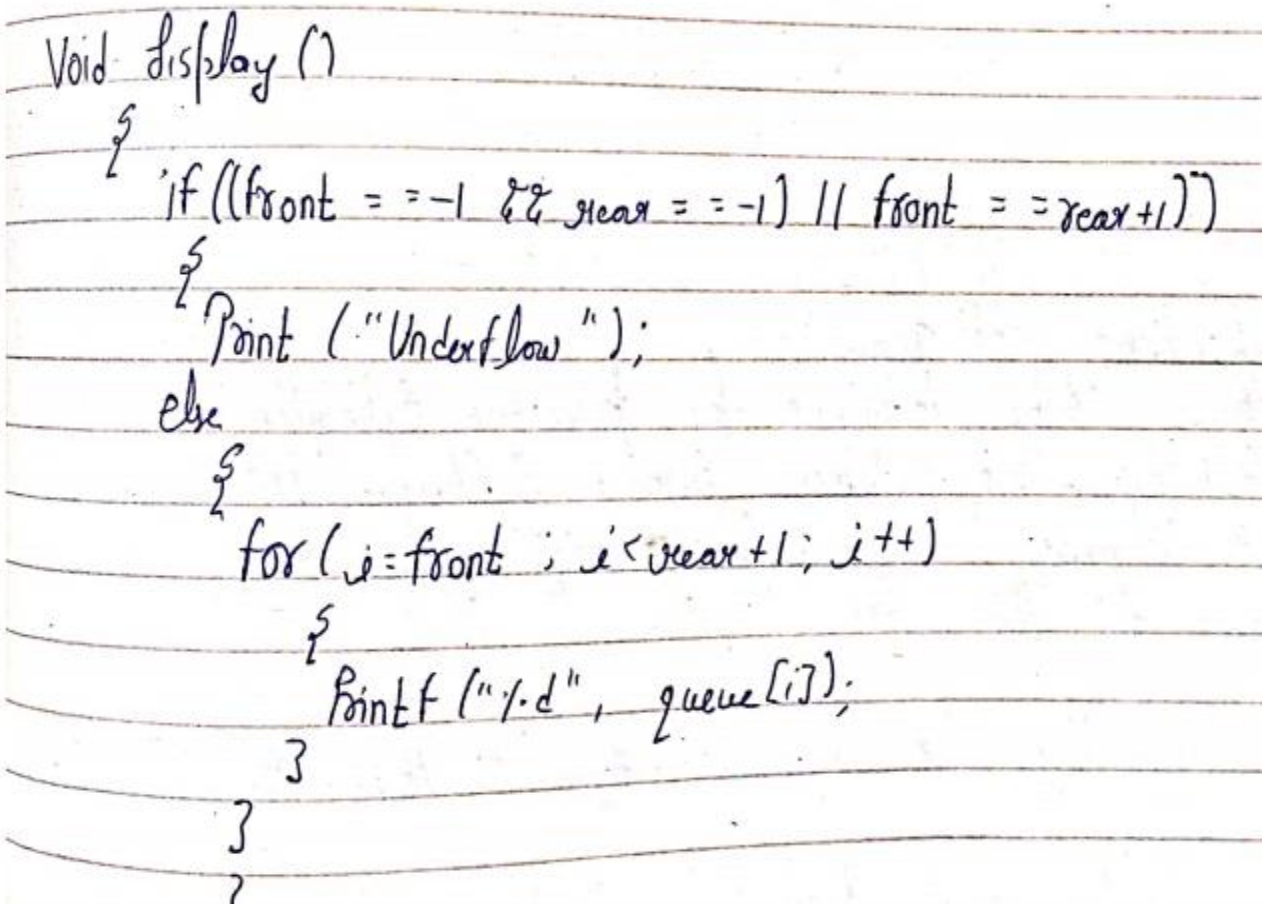
```
#define N 5
int queue[N];
int front = -1;
int rear = -1;
Void Enqueue(int x)
  if( rear = = N-1)
    {
      Printf ("Overflow");
    }
else if ( front = = -1 && rear = = -1)
      {   front = rear = 0;
else  }     queue[rear] = x;

else
      {
          rear++;
    queue [rear] = x;
      }
}
```

Queue Deletion

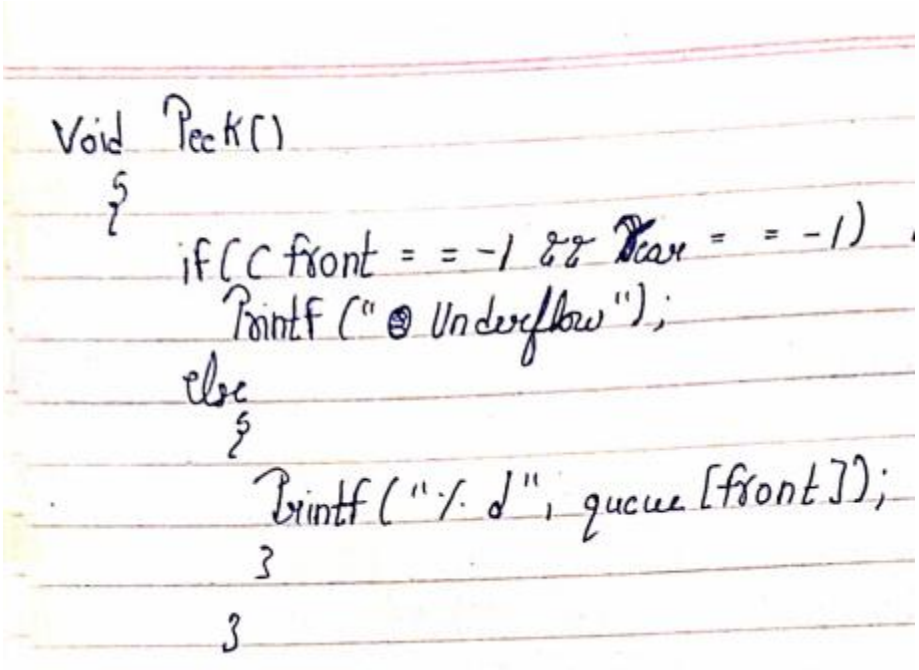
```
Void dequeue ()
{
  if ((front == -1 && rear == -1) ||
     {
       Print ("Underflow");
     }
  else if (front == rear)
     {
       front = rear = -1;
     }
  else
     { Printf ("%d", & queue[front]);
       front ++;
     }
}
```

Queue display



```
Void display ()
{
  if ((front == -1 && rear == -1) || front == rear+1))
  {
    Print ("Underflow");
  else
    {
      for (i=front ; i<rear+1; i++)
        {
          PrintF ("%d", queue[i];
        }
    }
}
```

Queue Peek



```
Void Peek()
{
    if(C front == -1 && rear == -1)
      Printf("@ Underflow");
    else
      {
        Printf("%d", queue[front]);
      }
}
```

Time and Space Complexity

Time Complexity

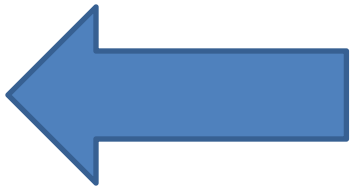
enqueue(),dequeue(),Peek() can be perfromed in constant time. So time Complexity of enqueue, dequeue() , Peek() are O(1) but display will print all the elements of queue. So its time complexity is O(N). Overall worst case time complexity of implementation of queue using array is O(N).

Space Complexity

As we are using an array of size n to store all the elements of the queue so space complexity using array implemenation is O(N).

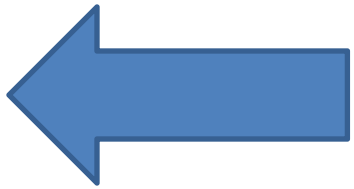
Linear Queue Implementation in C

```
#include <stdio.h>
#include<stdlib.h>
# define SIZE 100
void enqueue();
void dequeue();
void show();
int inp_arr[SIZE];
int Rear = - 1;
int Front = - 1;
int main()
{
   int ch;
   while (1)
   {
      printf("1.Enqueue Operation\n");
      printf("2.Dequeue Operation\n");
      printf("3.Display the Queue\n");
      printf("4.Exit\n");
      printf("Enter your choice of operations : ");
      scanf("%d", &ch);
```



Linear Queue Implementation in C

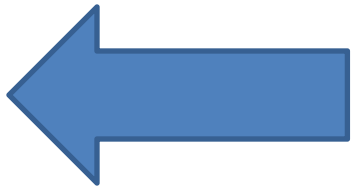
```
switch (ch)
      {
         case 1:
         enqueue();
         break;
         case 2:
         dequeue();
         break;
         case 3:
         show();
         break;
         case 4:
         exit(0);
         default:
         printf("Incorrect choice \n");
      }
   }
}
```



Linear Queue Implementation in C

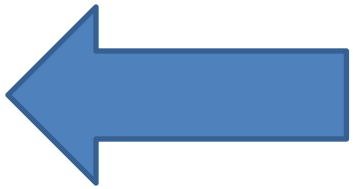
```
void enqueue()
{
    int insert_item;
    if (Rear == SIZE - 1)
       printf("Overflow \n");
    else if (Front == - 1 && Rear==-1){
        Front = 0, Rear=0;

        printf("Element to be inserted in the Queue\n : ");
        scanf("%d", &insert_item);
        inp_arr[Rear] = insert_item; }
else{
 printf("Element to be inserted in the Queue\n : ");
  Rear = Rear + 1;
        scanf("%d", &insert_item);
        inp_arr[Rear] = insert_item;
             }
}
```



Linear Queue Implementation in C

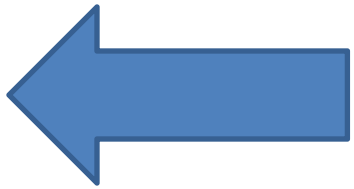
```
 void dequeue()
{
   if (Front == - 1 || Rrear==-1)
   {
      printf("Underflow \n");
      return ;
   }
   else
   {
      printf("Element deleted from the Queue: %d\n", inp_arr[Front]);
      Front = Front + 1;
   }
}
```



Linear Queue Implementation in C

```
void show()
{

   if (Front == - 1 && Rear==-1)
      printf("Empty Queue \n");
   else
   {
      printf("Queue: \n");
      for (int i = Front; i <= Rear; i++)
         printf("%d ", inp_arr[i]);
      printf("\n");
   }
}
```

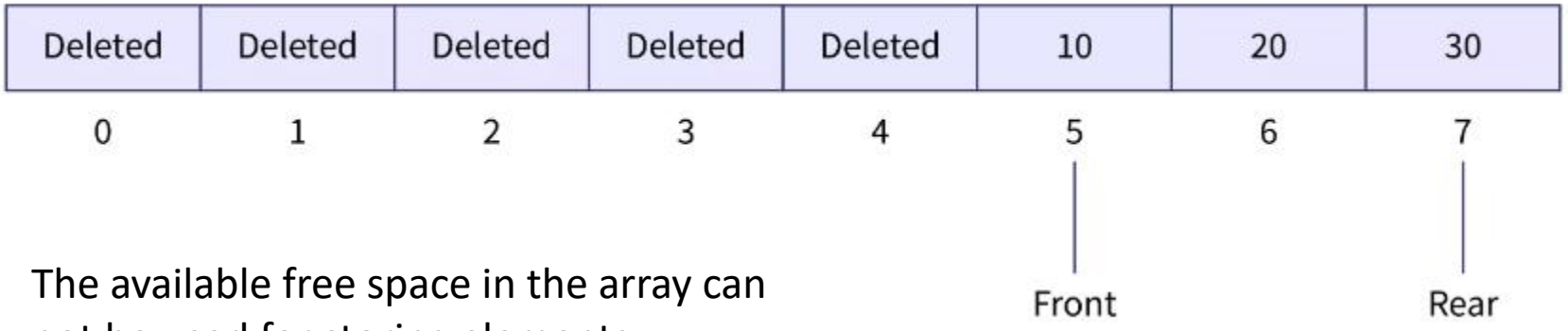


Drawback of Linear Queue

Once an element is deleted, we cannot insert another element in its position. This disadvantage of a linear queue is overcome by a circular queue.

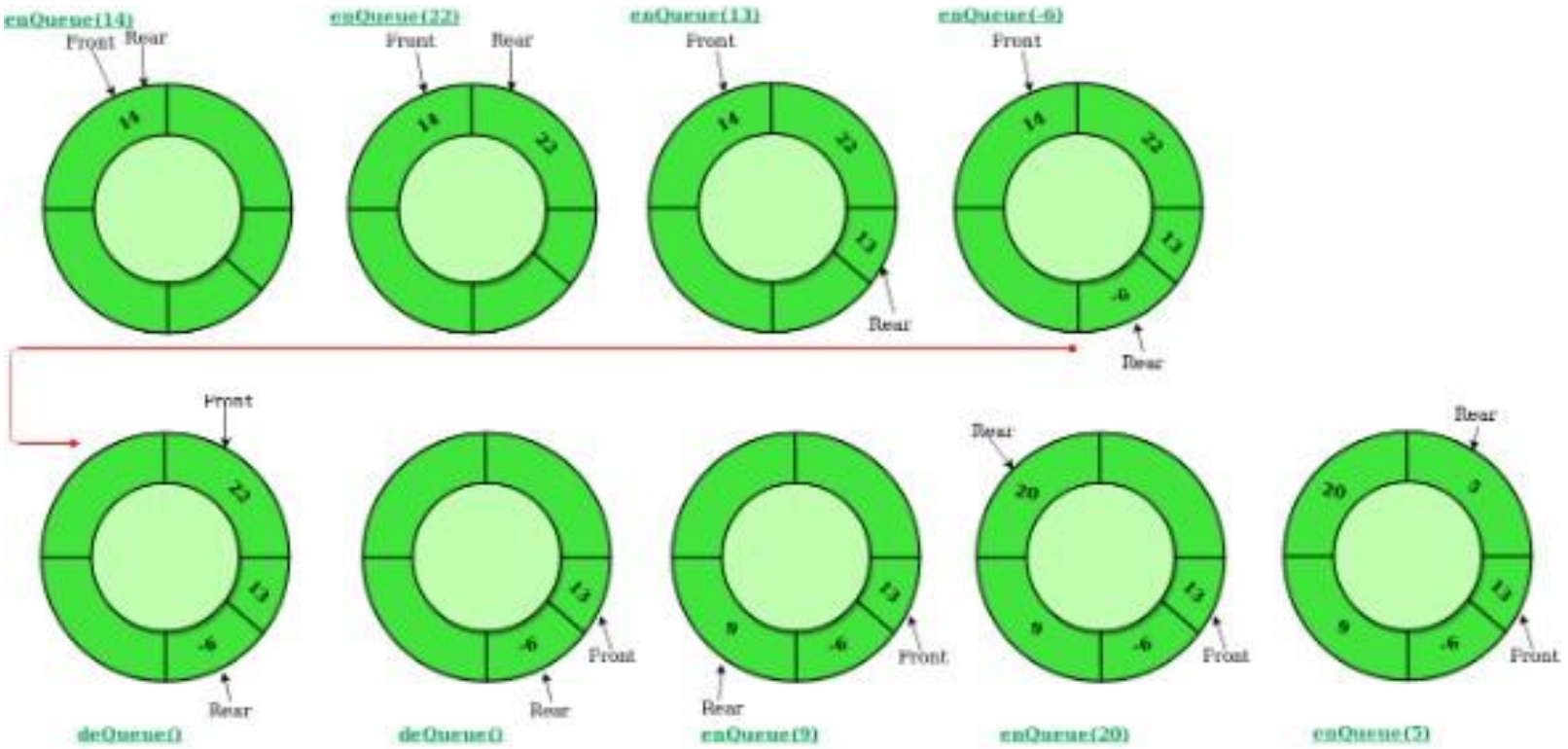
Example

- The major drawback of using a linear Queue is that insertion is done only from the rear end. If the first three elements are deleted from the Queue, we cannot insert more elements even though the space is available in a Linear Queue.
- In this case, the linear Queue shows the overflow condition as the rear is pointing to the last element of the Queue.



The available free space in the array can not be used for storing elements.

Circular Queue



Working of Circular queue operations

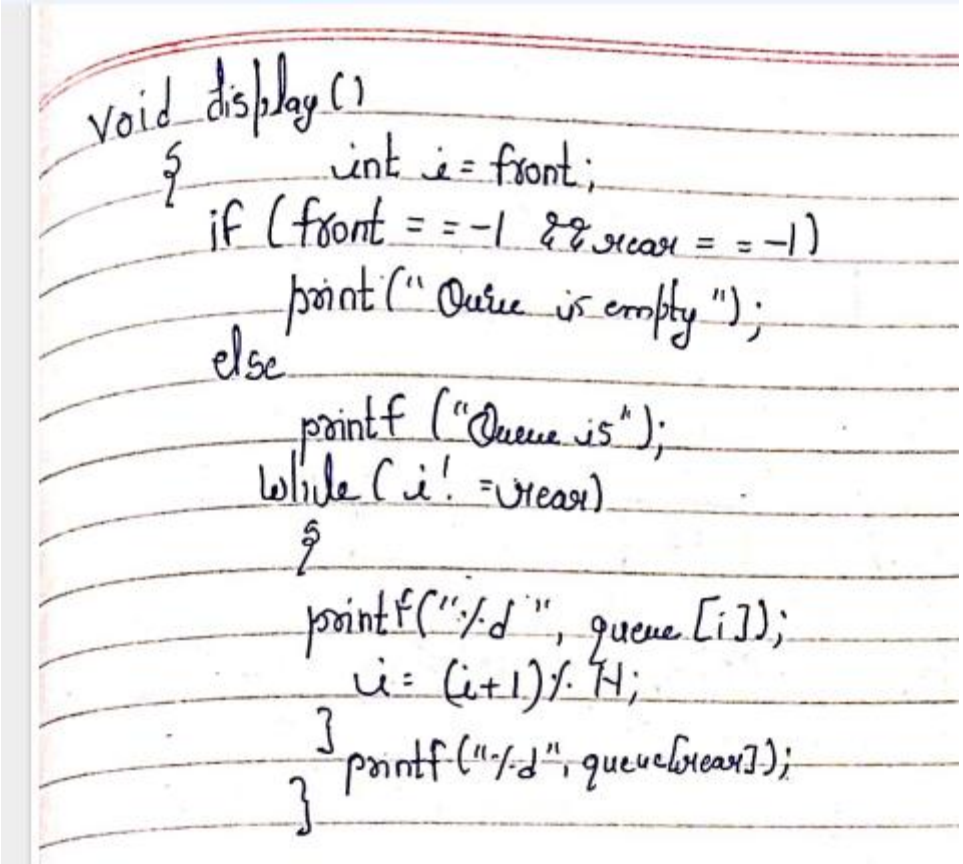
Insertion on Circular Queue

```
Void enqueue(int x)
{
  if (front == -1 && rear == -1)
  {
    front = rear = 0;
      queue[rear] = x;
  }
 else if((rear+1)%N) == front)
      {
        printf ("queue is full)
 else
     {
       rear = (rear+1)%N;
       queue[rear] = x;
   }
 }
```

Deletion from Circular queue

```
Void dequeue ()
{
  if ( front == -1 && rear == -1)
  {
    printf (" underflow")
  }
  else if (front == rear)
       {
         front = rear = -1;
       }
  else
    { Printf ("%d", queue [front])
      front = (front +1)%N;
    }
}
```

Display Circular queue



```
void display()
{
    int i = front;
    if (front == -1 && rear == -1)
        print("Queue is empty");
    else
        printf("Queue is");
    while (i != rear)
    {
        printf("%d", queue[i]);
        i = (i+1) % N;
    }
    printf("%d", queue[rear]);
}
```

Circular queue

Time Complexity

enqueue(),dequeue() can be perfromed in constant time. So time Complexity of enqueue, dequeue() are O(1) but display will print all the elements of circular queue. So its time complexity is O(N). Overall worst case time complexity of implementation of circular queue using array is O(N).

Space Complexity

As we are using an array of size n to store all the elements of the queue so space complexity using array implemenation is O(N).

Circular Queue Implementation in C

```
#include <stdio.h>

 define max 6

int queue[max];  // array declaration

int front=-1;

int rear=-1;

// function to insert an element in a circular queue
```

VIPS Vivekananda Institute of Professional Studies
योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

Circular Queue Implementation in C

```
void enqueue(int element)
{
   if(front==-1 && rear==-1)   // condition to check queue is empty
   {
      front=0;
      rear=0;
      queue[rear]=element;
   }
   else if((rear+1)%max==front)  // condition to check queue is full
   {
      printf("Queue is overflow..");
   }
   else
   {
      rear=(rear+1)%max;      // rear is incremented
      queue[rear]=element;     // assigning a value to the queue at the rear position.
   }

}
```

VIPS Vivekananda Institute of Professional Studies
योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

Circular Queue Implementation in C

```
void enqueue(int element)
{
   if(front==-1 && rear==-1)   // condition to check queue is empty
   {
      front=0;
      rear=0;
      queue[rear]=element;
   }
   else if((rear+1)%max==front)  // condition to check queue is full
   {
      printf("Queue is overflow..");
   }
   else
   {
      rear=(rear+1)%max;      // rear is incremented
      queue[rear]=element;     // assigning a value to the queue at the rear position.
   }

}
```

Circular Queue Implementation in C

```
void dequeue()
{
   if((front==-1) && (rear==-1))  // condition to check queue is empty
   {
      printf("\nQueue is underflow..");
   }
 else if(front==rear)
{
  printf("\nThe dequeued element is %d", queue[front]);
  front=-1;
  rear=-1;
}
else
{
   printf("\nThe dequeued element is %d", queue[front]);
  front=(front+1)%max;
}
}
```

Circular Queue Implementation in C

```
void display()
{
   int i=front;
   if(front==-1 && rear==-1)
   {
      printf("\n Queue is empty..");
   }
   else
   {
      printf("\nElements in a Queue are :");
      while(i<=rear)
      {
         printf("%d,", queue[i]);
         i=(i+1)%max;
      }
   }
}
```

Circular Queue Implementation in C

```
int main()
{
   int choice=1,x;   // variables declaration

   while(choice<4 && choice!=0)   // while loop
   {
   printf("\n Press 1: Insert an element");
   printf("\nPress 2: Delete an element");
   printf("\nPress 3: Display the element");
   printf("\nEnter your choice");
   scanf("%d", &choice);

   switch(choice)
   {

      case 1:

      printf("Enter the element which is to be inserted");
      scanf("%d", &x);
      enqueue(x);
      break;
      case 2:
      dequeue();
      break;
      case 3:
      display();

   }}
   return 0;
}
```

Deque

A deque is a list in which the elements can be inserted or deleted at either end.

Though the insertion and deletion in a deque can be performed on both ends, it does not follow the FIFO rule. The representation of a deque is given as follows -

Representation of deque

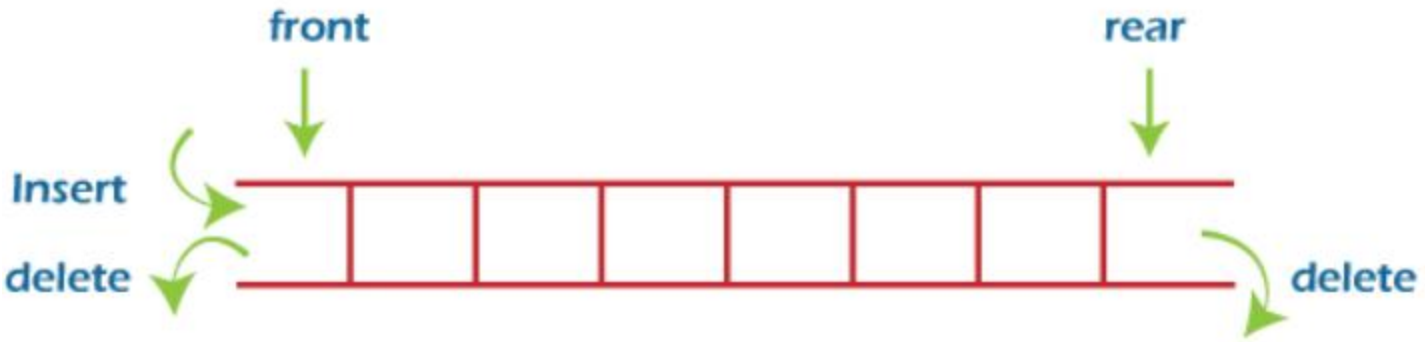
Deque

There are two types of deque -

- Input restricted queue
- Output restricted queue

Input restricted Queue

In input restricted queue, insertion operation can be performed at only one end, while deletion can be performed from both ends.

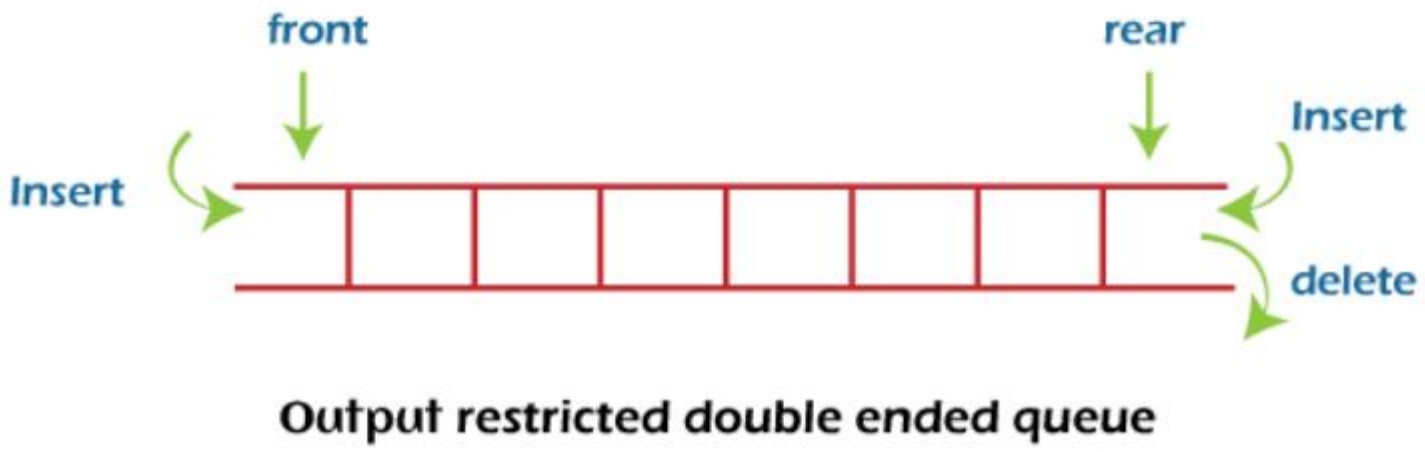


input restricted double ended queue

Deque

Output restricted Queue

In output restricted queue, deletion operation can be performed at only one end, while insertion can be performed from both ends.



Output restricted double ended queue

Priority Queues

A priority queue is a data structure in which each element is assigned a priority. The priority of the element will be used to determine the order in which the elements will be processed. The general rules of processing the elements of a priority queue are

- An element with higher priority is processed before an element with a lower priority.
- Two elements with the same priority are processed on a first-come-first-served (FCFS) basis.

Linked List Implementation of Priority Queue

- **Using Sorted list**: While a sorted list takes O(n) time to insert an element in the list, it takes only O(1) time to delete an element

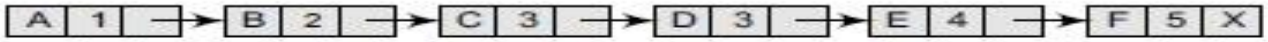


Figure 8.25 Priority queue

- **Using Unsorted list:** an unsorted list will take O(1) time to insert an element and O(n) time to delete an element from the list.

Array Representation of Priority Queue

- When arrays are used to implement a priority queue, then a separate queue for each priority number is maintained. Each of these queues will be implemented using circular arrays or circular queues. Every individual queue will have its own FRONT and REAR .

FRONT	REAR
3	3
1	3
4	5
4	1

	1	2	3	4	5
1			A		
2	B	C	D		
3				E	F
4	I			G	H

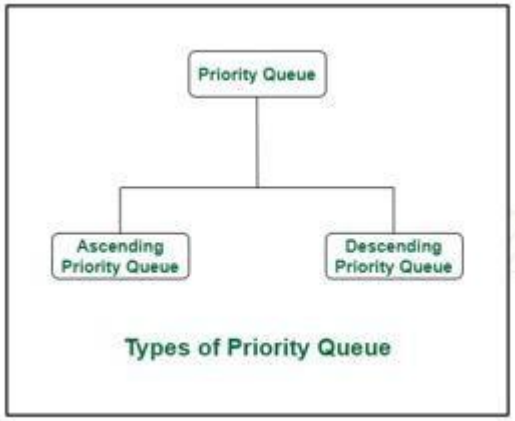
Figure 8.29 Priority queue matrix

FRONT	REAR
3	3
1	3
4	1
4	1

	1	2	3	4	5
1			A		
2	B	C	D		
3	R			E	F
4	I			G	H

Figure 8.30 Priority queue matrix after insertion of a new element

•If two elements in the queue have the same priority value then the first in first out rule is followed for these two elements alone i.e. the element that entered the priority queue first will be the first to be removed.



Types of Priority Queue

•**Complexity of Priority queue**

- Insertion : O(log n)$O(logn)$
- Peek : O(1)$O(1)$
- Deletion : O(log n)$O(logn$

Algorithm 6.19: This algorithm deletes and processes the first element in a priority queue maintained by a two-dimensional array QUEUE.

1. [Find the first nonempty queue.]
   Find the smallest K such that FRONT[K] ≠ NULL.
2. Delete and process the front element in row K of QUEUE.
3. Exit.

Algorithm 6.20: This algorithm adds an ITEM with priority number M to a priority queue maintained by a two-dimensional array QUEUE.

1. Insert ITEM as the rear element in row M of QUEUE.
2. Exit.

https://www.youtube.com/watch?v=ejJ5MjARAUM&ab_channel=TheCoderHuman

The main difference between a queue and a priority queue:

The element that was added first to the queue will be the first to be removed. However, in a priority queue, the element with the highest priority will be the first to be dequeued.

The outcome of an element being removed from the priority queue will be in sorted order, which may be growing or decreasing. When in a queue, elements are released in FIFO (First in First out) order.

END